Matched Filtering for Gravitational Wave Detection

Matched filtering is the primary technique for detecting gravitational wave signals in noisy detector data. It correlates known template waveforms with the detector data to find signals with high signal-to-noise ratio (SNR).

Overview

Matched filtering requires:
1. Template waveform (expected signal shape)
2. Conditioned detector data (preprocessed strain)
3. Power spectral density (PSD) of the noise
4. SNR calculation and peak finding

PyCBC supports both time-domain and frequency-domain approaches.

Time-Domain Waveforms

Generate templates in time domain using `get_td_waveform`:

```python
from pycbc.waveform import get_td_waveform
from pycbc.filter import matched_filter

# Generate time-domain waveform
hp, hc = get_td_waveform(
    approximant='IMRPhenomD',  # or 'SEOBNRv4_opt', 'TaylorT4'
    mass1=25,                  # Primary mass (solar masses)
    mass2=20,                  # Secondary mass (solar masses)
    delta_t=conditioned.delta_t,  # Must match data sampling
    f_lower=20                 # Lower frequency cutoff (Hz)
)

# Resize template to match data length
hp.resize(len(conditioned))

# Align template: cyclic shift so merger is at the start
template = hp.cyclic_time_shift(hp.start_time)
```

```
# Perform matched filtering
snr = matched_filter(
    template,
    conditioned,
    psd=psd,
    low_frequency_cutoff=20
)

# Crop edges corrupted by filtering
# Remove 4 seconds for PSD + 4 seconds for template length at start
# Remove 4 seconds at end for PSD
snr = snr.crop(4 + 4, 4)

# Find peak SNR
import numpy as np
peak_idx = np.argmax(abs(snr).numpy())
peak_snr = abs(snr[peak_idx])
```

Why Cyclic Shift?

Waveforms from `get_td_waveform` have the merger at time zero. For matched filtering, we typically want the merger aligned at the start of the template. `cyclic_time_shift` rotates the waveform appropriately.

Frequency-Domain Waveforms

Generate templates in frequency domain using `get_fd_waveform`:

```python
from pycbc.waveform import get_fd_waveform
from pycbc.filter import matched_filter

# Calculate frequency resolution
delta_f = 1.0 / conditioned.duration

# Generate frequency-domain waveform
hp, hc = get_fd_waveform(
    approximant='IMRPhenomD',
    mass1=25,
    mass2=20,
    delta_f=delta_f,          # Frequency resolution (must match data)
    f_lower=20                # Lower frequency cutoff (Hz)
)

# Resize template to match PSD length
hp.resize(len(psd))

# Perform matched filtering
snr = matched_filter(
    hp,
    conditioned,
    psd=psd,
    low_frequency_cutoff=20
)
```

```
# Find peak SNR
import numpy as np
peak_idx = np.argmax(abs(snr).numpy())
peak_snr = abs(snr[peak_idx])
```

Key Differences: Time vs Frequency Domain

Time Domain (`get_td_waveform`)
- **Pros**: Works for all approximants, simpler to understand
- **Cons**: Can be slower for long waveforms
- **Use when**: Approximant doesn't support frequency domain, or you need time-domain manipulation

Frequency Domain (`get_fd_waveform`)
- **Pros**: Faster for matched filtering, directly in frequency space
- **Cons**: Not all approximants support it (e.g., SEOBNRv4_opt may not be available)
- **Use when**: Approximant supports it and you want computational efficiency

Approximants

Common waveform approximants:

```python
# Phenomenological models (fast, good accuracy)
'IMRPhenomD'      # Good for most binary black hole systems
'IMRPhenomPv2'    # More accurate for precessing systems

# Effective One-Body models (very accurate, slower)
'SEOBNRv4_opt'    # Optimized EOB model (time-domain only typically)

# Post-Newtonian models (approximate, fast)
'TaylorT4'        # Post-Newtonian expansion
```

Note: Some approximants may not be available in frequency domain. If `get_fd_waveform` fails, use `get_td_waveform` instead.

Matched Filter Parameters

`low_frequency_cutoff`
- Should match your high-pass filter cutoff (typically 15-20 Hz)
- Templates are only meaningful above this frequency
- Lower values = more signal, but more noise

Template Resizing
- **Time domain**: `hp.resize(len(conditioned))` - match data length
- **Frequency domain**: `hp.resize(len(psd))` - match PSD length
- Critical for proper correlation

Crop Amounts
After matched filtering, crop edges corrupted by:
- **PSD filtering**: 4 seconds at both ends
- **Template length**: Additional 4 seconds at start (for time-domain)
- **Total**: `snr.crop(8, 4)` for time-domain, `snr.crop(4, 4)` for frequency-domain

Best Practices

1. **Match sampling/frequency resolution**: Template `delta_t`/`delta_f` must match data
2. **Resize templates correctly**: Time-domain → data length, Frequency-domain → PSD length
3. **Crop after filtering**: Always crop edges corrupted by filtering
4. **Use `abs()` for SNR**: Matched filter returns complex SNR; use magnitude
5. **Handle failures gracefully**: Some approximants may not work for certain mass combinations

Common Issues

Problem: "Approximant not available" error
- **Solution**: Try time-domain instead of frequency-domain, or use different approximant

Problem: Template size mismatch
- **Solution**: Ensure template is resized to match data length (TD) or PSD length (FD)

Problem: Poor SNR even with correct masses
- **Solution**: Check that PSD low_frequency_cutoff matches your high-pass filter, verify data conditioning

Problem: Edge artifacts in SNR time series
- **Solution**: Increase crop amounts or verify filtering pipeline order

Dependencies

```bash
pip install pycbc numpy
```

References

- [PyCBC Tutorial: Waveform & Matched Filter](https://colab.research.google.com/github/gwastro/pycbc-tutorials/blob/master/tutorial/3_WaveformMatchedFilter.ipynb) - Comprehensive tutorial demonstrating waveform generation, matched filtering in both time and frequency domains, and SNR analysis
- [PyCBC Waveform Documentation](https://pycbc.org/pycbc/latest/html/pycbc.waveform.html)
- [PyCBC Filter Documentation](https://pycbc.org/pycbc/latest/html/pycbc.filter.html)